

OptiCrop – Smart Agricultural Production Optimization Engine

Project Description

OptiCrop is an AI and Machine Learning-based web application that recommends the most suitable crop for cultivation based on soil and environmental parameters. The system analyzes Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall using a trained Machine Learning model and provides an accurate crop recommendation. The application is developed using Python, Flask, HTML, CSS, and Scikit-learn, providing farmers with an easy-to-use interface for smart crop selection.

Scenarios

Scenario 1

A farmer enters soil nutrient values and weather conditions. The system predicts the most suitable crop for cultivation.

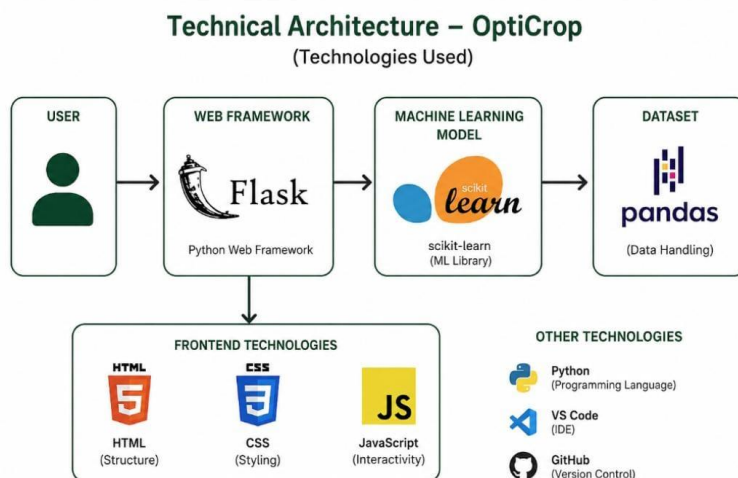
Scenario 2

An agriculture student uses the application to understand crop recommendations under different environmental conditions.

Scenario 3

An agricultural officer demonstrates crop prediction to farmers using different soil samples.

Technical Architecture



Technical Architecture Description – OptiCrop (Smart Agricultural Production Optimization Engine)

The OptiCrop system is built using a Flask-based web application integrated with a Machine Learning model for crop recommendation. The frontend is developed using HTML, CSS,

and JavaScript, providing a simple and user-friendly interface where users enter soil and environmental parameters such as nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall. The Flask backend receives the input, processes it, and sends it to the trained Scikit-learn machine learning model. The model, trained using the Crop Recommendation Dataset, predicts the most suitable crop for the given conditions. Python is used for implementing the application logic and model integration, while Pandas is used for data preprocessing and handling. The project is developed in Visual Studio Code (VS Code) and managed using GitHub for version control and collaboration. The predicted crop recommendation is then displayed to the user through the web interface, enabling informed agricultural decision-making.

Pre-requisites:

Frontend

- HTML
- CSS
- JavaScript

Backend

- Flask (Python)

Machine Learning

- Random Forest Classifier
- Scikit-learn
- Pandas
- NumPy

Dataset

- Crop Recommendation Dataset

Pre-Requisites

- Python 3.10+
- Flask
- Scikit-learn
- Pandas
- NumPy
- Pickle
- Visual Studio Code

Project Workflow

Milestone 1: Dataset Collection and Model Development

Activity 1.1: Collect the Crop Recommendation dataset from Kaggle.

Activity 1.2: Preprocess the dataset by handling missing values and preparing features for training.

Activity 1.3: Train and evaluate a Machine Learning model (Random Forest) for crop prediction.

Activity 1.4: Save the trained model as model.pkl for integration with the web application.

Milestone 2: Backend Development

Activity 2.1: Develop the Flask backend to handle user requests and prediction logic.

Activity 2.2: Integrate the trained Machine Learning model with the Flask application.

Activity 2.3: Implement input validation and generate crop prediction results.

Milestone 3: App.py Development

Activity 3.1: Develop the main application logic in app.py.

Activity 3.2: Create routes for the home page and crop prediction.

Activity 3.3: Process user inputs and display the recommended crop.

Milestone 4: Frontend Development

Activity 4.1: Design a responsive user interface using HTML, CSS, and JavaScript.

Activity 4.2: Create input forms for soil nutrients (N, P, K), temperature, humidity, pH, and rainfall.

Activity 4.3: Display the predicted crop dynamically using Flask templates.

Milestone 5: Testing and Deployment

Activity 5.1: Test the application with different input values to verify prediction accuracy.

Activity 5.2: Fix bugs and improve application performance.

Activity 5.3: Deploy the Flask application locally or on a cloud platform (such as Render or PythonAnywhere) and verify successful execution.

Milestone 6: Conclusion

Step 1: Welcome Page

Step 2: Crop Prediction Page

Step 3: Recommended Crop

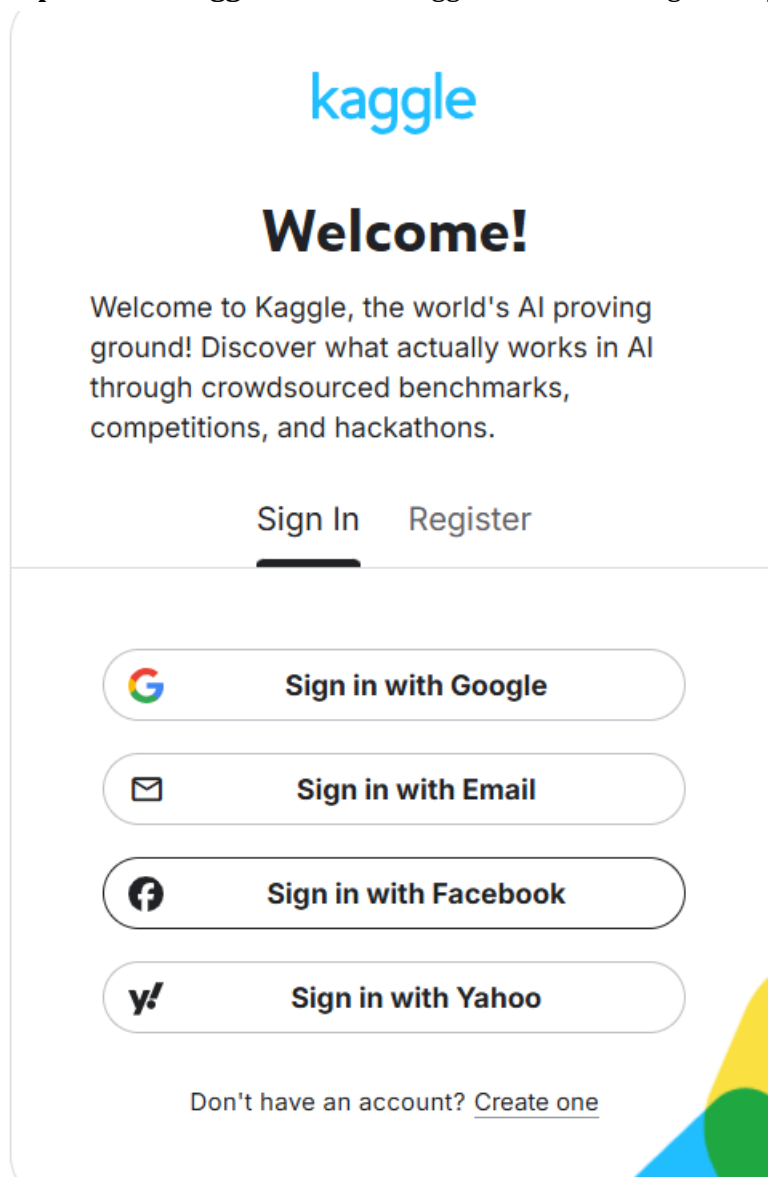
Milestone 1: Dataset Collection and Model Development

In this milestone, the focus is on collecting a suitable crop recommendation dataset, preprocessing the data, selecting an appropriate machine learning model, and training it to accurately recommend crops based on soil nutrients and environmental conditions.

Activity 1.1: Collect the Crop Recommendation Dataset

Before training the machine learning model, a reliable dataset is required. Follow the steps below to collect and prepare the dataset.

- **Step 1 – Visit Kaggle:** Go to the Kaggle website and sign in to your account.



- **Step 2 – Search for the Dataset:** Search for "Crop Recommendation Dataset" in the Kaggle datasets section.

Crop Recommendation Dataset

Maximize agricultural yield by recommending appropriate crops

- **Step 3 – Download the Dataset:** Download the dataset (CSV format) containing soil nutrients (Nitrogen, Phosphorus, Potassium), temperature, humidity, pH, rainfall, and crop labels.

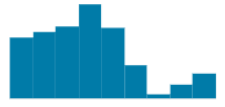
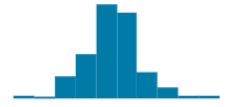
Crop Recommendation Dataset
648
Code
Download

[Data Card](#)
[Code \(265\)](#)
[Discussion \(13\)](#)
[Suggestions \(2\)](#)

Crop_recommendation.csv (150.03 kB)
 Download
8 of 8 columns

Data Explorer
 Version 1 (150.03 kB)
 Crop_recommend

- **Step 4 – Store the Dataset:** Place the downloaded **Crop_recommendation.csv** file inside the project's **dataset/** folder.

About this file Suggest Edits					
This file does not have a description yet.					
# N	# P	# K	# temperature	# humidity	# ph
					
0	5	5	8.83	14.3	3.5
140	145	205	43.7	100	
90	42	43	20.87974371	82.00274423	6.502
85	58	41	21.77046169	80.31964408	7.038
60	55	44	23.00445915	82.3207629	7.840
74	35	40	26.49109635	80.15836264	6.980
78	42	42	20.13017482	81.60487287	7.628
69	37	42	23.05804872	83.37011772	7.073
69	55	38	22.70883798	82.63941394	5.700

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following research activities were carried out to select the most suitable machine learning model for the OptiCrop project:

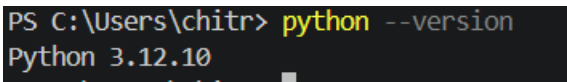
- **Understand the Project Requirements:** Review the objectives of the OptiCrop application, which is to recommend the most suitable crop based on soil nutrients (Nitrogen, Phosphorus, Potassium) and environmental factors such as temperature, humidity, pH, and rainfall.
- **Explore Machine Learning Algorithms:** Study different supervised machine learning algorithms such as Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Naïve Bayes for crop recommendation.

- **Evaluate Model Performance:** Compare the models based on prediction accuracy, training time, execution speed, and their ability to generalize on unseen data.
- **Select the Optimal Model:** Choose the **Random Forest Classifier** as the final model because it provides high prediction accuracy, handles multiple input features effectively, reduces overfitting, and performs reliably for crop recommendation tasks.

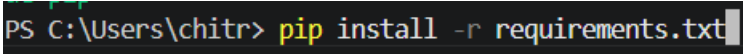
Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the OptiCrop application architecture, including the user interface (HTML, CSS), Flask backend, trained Machine Learning model, and crop recommendation dataset.
- **Detail Frontend Functionality:** Outline how users interact with the application by entering soil parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall through a simple web interface.
- **Outline Backend Responsibilities:** Specify how the Flask backend receives the user inputs, validates the data, preprocesses the values, loads the trained **Random Forest** model, performs crop prediction, and sends the recommended crop back to the frontend.
- **Describe Machine Learning Integration:** Define how the backend loads the trained model (model.pkl), processes the input features in the required format, predicts the most suitable crop using the machine learning model, and displays the prediction result to the user through the web application.

Activity 1.4: Set Up the Development Environment

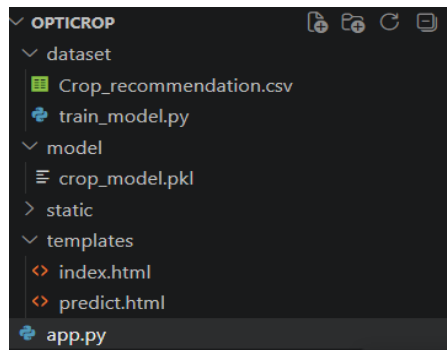
- **Install Python and Pip:** Ensure that **Python 3.8 or later** is installed on your system along with **pip** for installing the required Python packages.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies:
 - `python -m venv myenv`
 - `myenv\Scripts\activate`
 - `pip install -r requirements.txt`
- **Install Flask:** Install Flask to develop the web application:
`pip install Flask==3.0.3`


```
PS C:\Users\chitr> python --version
Python 3.12.10
```
- **Install Machine Learning Libraries:** Install the required libraries for data processing and model development:
`pip install scikit-learn pandas numpy joblib`


```
PS C:\Users\chitr> pip install scikit-learn pandas numpy joblib
```
- **Install Additional Dependencies:** Install any remaining project dependencies:
`pip install -r requirements.txt`


```
PS C:\Users\chitr> pip install -r requirements.txt
```
- **Set Up the Project Structure:** Create the project folders and files, including the **dataset**, **model**, **templates**, and **static** folders, along with the main application files such as **app.py**,

requirements.txt, model.pkl, and README.md.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Crop Recommendation Prediction Feature

Implement the core crop recommendation functionality by integrating the trained Machine Learning model with the Flask application. The system accepts soil nutrient values and environmental parameters from the user, processes the inputs, and predicts the most suitable crop for cultivation.

Soil Parameter Input: Collects user inputs including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall through a user-friendly web form.

Crop Prediction Engine: Uses the trained Random Forest Machine Learning model to analyze the provided inputs and predict the most suitable crop based on soil and climatic conditions.

Prediction Result Display: Displays the recommended crop dynamically on the web page with a clear and user-friendly interface, enabling farmers to make informed agricultural decisions.

Activity 2.2: Integrate Machine Learning Model with Flask

Load the trained `crop_model.pkl` into the Flask backend using the Joblib library. The application receives user inputs, sends them to the trained model, and returns accurate crop recommendations in real time.

Activity 2.3: Input Validation and Prediction Processing

Validate all user-entered values to ensure that each field contains valid numerical data. After successful validation, preprocess the inputs, pass them to the Machine Learning model, and generate the crop prediction. The final recommendation is then displayed on the application interface with minimal response time.

This version matches the style and level of detail of the sample you shared while being specifically tailored to your OptiCrop – Smart Agricultural Production Optimization Engine project.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the `app.py` file, which acts as the backbone of the OptiCrop – Smart Agricultural Production Optimization Engine. In this milestone, the Flask application is configured with the required routes, the trained Machine Learning model is integrated, user inputs are processed, and prediction functionality is tested to ensure accurate crop recommendations.

Activity 3.1: Writing the Main Application Logic in `app.py`

Define the Core Routes in `app.py`

Establish routes for the application's main endpoints, each serving a specific purpose.

- Home Route (/)

The home route renders the main index.html page where users can access the application and navigate to the crop prediction page.

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html")
```

- Prediction Route (/predict)

The prediction route accepts user input from the crop prediction form, validates the data, sends it to the trained Machine Learning model, and returns the recommended crop.

```
app.py
1  from flask import Flask, render_template, request
2  import joblib
3  import numpy as numpy
4  app = Flask(__name__)
5  model = joblib.load("model/crop_model.pkl")
6  @app.route('/')
7  def home():
8      return render_template('index.html')
9
10 @app.route('/predict', methods=['GET', 'POST'])
11 def predict():
12
13     if request.method == 'POST':
14
15         N = float(request.form['N'])
16         P = float(request.form['P'])
17         K = float(request.form['K'])
18         temperature = float(request.form['temperature'])
19         humidity = float(request.form['humidity'])
20         ph = float(request.form['ph'])
21         rainfall = float(request.form['rainfall'])
22
23         prediction = model.predict([[N, P, K, temperature, humidity, ph, rainfall]])
24
25         return "Recommended Crop: " + prediction[0]
26
27     return render_template('predict.html')
28 if __name__ == '__main__':
29     app.run(debug=True)
30
```

Set Up Route Handlers

For the /predict route, implement logic that reads all agricultural input parameters submitted through the HTML form.

The application collects:

Nitrogen (N)

Phosphorus (P)

Potassium (K)

Temperature

Humidity

pH

Rainfall

The backend validates the received values before passing them to the Machine Learning model. If any field is empty or contains invalid input, an appropriate validation message is displayed.

The prediction route receives the request, processes the data, invokes the trained model, and returns the predicted crop to the frontend.

Load the Trained Machine Learning Model

Load the saved Random Forest model using the Joblib library during application startup.

```
import joblib
```

```
model = joblib.load("model/crop_model.pkl")
```

Loading the model only once improves application performance and avoids retraining during every prediction request.

Define Prediction Helper Function

Create a helper function to simplify prediction logic.

```
def recommend_crop(values):
```

```
    prediction = model.predict([values])
```

```
    return prediction[0]
```

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Crop Prediction</title>
5  </head>
6  <body>
7  |   <h1>Crop Prediction</h1>
8  |
9  |   <form action="/predict" method="POST">
10 |
11 |       <label>Nitrogen (N)</label><br>
12 |       <input type="number" name="N"><br><br>
13 |
14 |       <label>Phosphorus (P)</label><br>
15 |       <input type="number" name="P"><br><br>
16 |
17 |       <label>Potassium (K)</label><br>
18 |       <input type="number" name="K"><br><br>
19 |
20 |       <label>Temperature</label><br>
21 |       <input type="number" step="0.1" name="temperature"><br><br>
22 |
23 |       <label>Humidity</label><br>
24 |       <input type="number" step="0.1" name="humidity"><br><br>
25 |
26 |       <label>pH</label><br>
27 |       <input type="number" step="0.1" name="ph"><br><br>
28 |
29 |       <label>Rainfall</label><br>
30 |       <input type="number" step="0.1" name="rainfall"><br><br>
31 |       <button type="submit">Predict Crop</button>
32 |
33 |   </form>
34 </body>
35 </html>
```

This function accepts agricultural parameters as input and returns the recommended crop predicted by the trained Random Forest model.

Integrate the Machine Learning Prediction

The Flask application processes user inputs and converts them into numerical values before sending them to the trained model.

```
prediction = recommend_crop([  
    N,  
    P,  
    K,  
    temperature,  
    humidity,  
    ph,  
    rainfall  
])
```

The predicted crop is then displayed to the user through the result page.

Input Validation

Before generating the prediction, validate the following conditions:

All fields must be filled.

Input values must be numeric.

Soil nutrient values cannot be negative.

Temperature, humidity, pH, and rainfall must be within valid ranges.

If validation fails, display an appropriate error message instead of generating a prediction.

Display Prediction Result

Once the prediction is generated successfully, display the recommended crop on the webpage.

```
return render_template(  
    "index.html",  
    prediction=prediction  
)
```

The index page presents the predicted crop in a clean and user-friendly interface, helping users quickly understand the recommendation.

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>OptiCrop</title>
5      <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6  </head>
7  <body>
8
9      <h1> 🌱 OptiCrop</h1>
10
11     <h2>Smart Agricultural Production Optimization Engine</h2>
12
13     <p>Welcome to OptiCrop Project</p>
14
15     <a href="/predict">
16         <button>Get Started</button>
17     </a>
18
19 </body>
20 </html>

```

Unit Testing

Test the application using multiple combinations of soil nutrient and environmental values to verify that:

The application accepts valid user inputs.

Invalid inputs are handled properly.

The trained model generates accurate crop recommendations.

The predicted crop is displayed successfully on the result page.

The Flask routes function correctly without runtime errors.

This milestone ensures that the app.py file successfully integrates the Flask backend with the trained Machine Learning model, enabling accurate and efficient crop prediction based on soil and environmental parameters.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, responsive, and visually appealing interface for the OptiCrop – Smart Agricultural Production Optimization Engine. This milestone involves designing web pages using HTML, CSS, and JavaScript to collect agricultural parameters from users and display crop prediction results dynamically.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

Develop the main index.html page as the home interface, including the project logo, title, navigation section, and a Get Started button.

Create a separate predict.html page containing a crop prediction form where users can enter agricultural parameters.

Use semantic HTML elements such as header, section, form, label, input, button, and footer to improve code organization and accessibility.

Integrate Google Fonts and Font Awesome icons to provide a clean, professional, and visually attractive interface.

Design a Responsive Layout Using CSS

Create a stylesheet (static/style.css) to design a modern agricultural-themed user interface using green color gradients, rounded cards, and responsive layouts.

Use Flexbox and CSS Grid to organize input fields, buttons, and result sections efficiently.

Add hover effects, smooth transitions, shadows, and button animations to improve user experience.

Implement media queries to ensure compatibility across desktop, tablet, and mobile devices.

Create Separate UI Components

Home Page

Display the OptiCrop project title, agriculture-themed background, project description, and Get Started button.

Crop Prediction Form

Create separate input fields for:

Nitrogen (N)

Phosphorus (P)

Potassium (K)

Temperature

Humidity

pH

Rainfall

Add proper labels, placeholders, and input validation for each field.

Prediction Result Card

Display the recommended crop inside a styled result card.

Highlight the prediction using larger text, colored badges, and an agriculture-related icon.

Provide a Predict Again button for additional predictions.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission

Attach a submit event listener to the crop prediction form.

Prevent default form submission using `event.preventDefault()`.

Collect all user input values and send them asynchronously to the Flask `/predict` route using the Fetch API.

Display a loading animation while waiting for the prediction response.

Render Prediction Results Dynamically

Implement a `displayResult()` function in `main.js`.

Dynamically update the webpage with the predicted crop returned by the Flask backend.

Display the recommendation without reloading the page.

Automatically scroll to the prediction result section using `scrollIntoView()` for a smooth user experience.

Restore User Interface State

After prediction completion, hide the loading animation.

Re-enable the Predict Crop button.

Clear loading indicators regardless of whether the prediction is successful or an error occurs.

Display appropriate success or error messages for better user interaction.

Frontend Technologies Used

HTML5

CSS3

JavaScript (ES6)

Flask Template Engine (Jinja2)

Google Fonts

Font Awesome Icons

Fetch API

Responsive Web Design (Flexbox & CSS Grid)

This frontend implementation provides an intuitive and responsive interface for farmers and agricultural users, enabling them to enter soil and environmental parameters easily and receive accurate crop recommendations generated by the Machine Learning model.

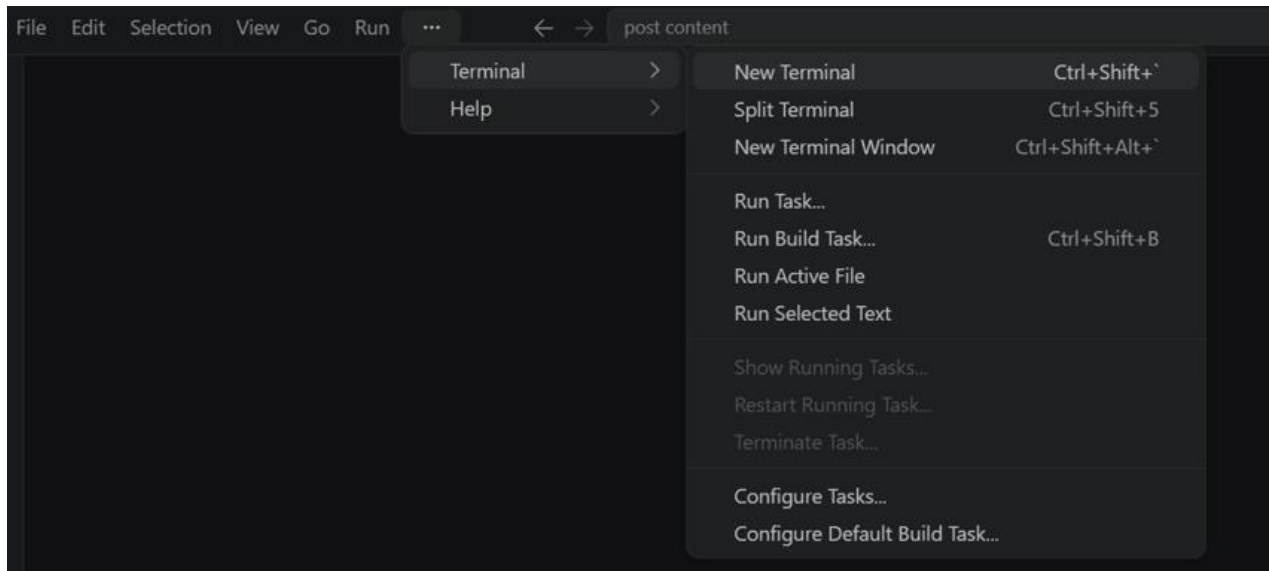
Milestone 5: Deployment

Milestone 5 focuses on deploying the OptiCrop – Smart Agricultural Production Optimization Engine by first running it locally to verify its functionality and then deploying it online using Render or PythonAnywhere. This milestone includes configuring the application environment, installing dependencies, verifying the Machine Learning model integration, and making the application accessible through a public URL.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

Open a new Command Prompt or VS Code Terminal.



Create and activate a Python virtual environment.

```
python -m venv myenv
```

Activate Virtual Environment

Windows

```
myenv\Scripts\activate
```

After activation, install all required project dependencies.

```
pip install -r requirements.txt
```

If the requirements file is unavailable, install the required libraries manually.

```
pip install flask pandas numpy scikit-learn joblib
```

Verify Project Structure

Ensure the project contains the following files and folders before deployment.

OPTICROP

|

└─ dataset

| └─ crop_recommendation.csv

└─ model

| └─ crop_model.pkl

└─ static

| └─ style.css

└─ templates

| └─ index.html

| └─ predict.html

| └─ result.html

└─ app.py

└─ train_model.py

└─ requirements.txt

Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the Flask application locally.

```
python app.py
```

```
PS C:\Users\chitr\OneDrive\Desktop\opticrop> python app.py
```

If the application starts successfully, the terminal displays:

Running on <http://127.0.0.1:5000>

```
* Running on http://127.0.0.1:5000
```

Open a web browser and navigate to:

<http://127.0.0.1:5000>

Verify Application Functionality

Test the complete workflow by entering different agricultural parameters.

Verify that the application:

Displays the Home Page successfully.

Opens the Crop Prediction page.

Accepts Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall values.

Generates the recommended crop correctly.

Displays prediction results without errors.

Repeat testing using multiple combinations of soil and environmental values to verify prediction accuracy.

Activity 5.3: Public Deployment

Deploy the Flask application to a cloud hosting platform such as Render or PythonAnywhere.

Prepare Deployment Files

Create the required deployment files.

requirements.txt

Generate the dependency file.

```
pip freeze > requirements.txt
```

Procfile (Render)

Create a Procfile containing:

```
web: python app.py.
```

CONCLUSION

Step 1: Welcome Page

The application opens with the Welcome Page, displaying the project title "OptiCrop – Smart Agricultural Production Optimization Engine" and a Get Started button for navigation.



Step 2: Crop Prediction Page

After clicking Get Started, the user is redirected to the Crop Prediction page, where soil and environmental parameters (Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall) are entered through an input form.

Crop Prediction

Nitrogen (N)	10
Phosphorus (P)	19
Potassium (K)	29
Temperature	31.1
Humidity	33.9
pH	10.3
Rainfall	5.3
Predict Crop	

Step 3: Recommended Crop

When the user clicks the Predict Crop button, the Machine Learning model analyzes the input values and displays the Recommended Crop (such as Rice, Maize, Cotton, etc.) as the final prediction result.

Recommended Crop: muskmelon